

Automated Documentation Workflow

Task 1 — Draw the Documentation Workflow

Objective:

Create a clear visual representation of the end-to-end documentation workflow.

What to do:

- Map the workflow phases:
 1. Scope identification
 2. File/logic analysis
 3. Inline documentation application
 4. Sphinx automation and generation
 5. Review and maintenance

Deliverable:

A workflow diagram (PNG, PDF, or editable source) with explanatory notes.

Acceptance criteria:

- All phases are represented.
 - Arrows show the sequence and dependencies.
 - The diagram clearly indicates where the documentation agent interacts with the workflow.
-

Task 2 — Integrate the Workflow into an Existing Documentation Agent

Objective:

Adapt an existing documentation agent to execute the workflow automatically.

What to do:

- Analyze the existing agent's current capabilities and architecture.
- Map each workflow phase to agent actions, prompts, or tool calls.
- Implement the workflow as a state machine, pipeline, or agent loop.
- Add error handling and logging for failures.

Deliverable:

Updated agent code/config, plus a short integration document explaining how the workflow is embedded.

Acceptance criteria:

- Running the agent on a sample codebase follows the documented workflow.
 - The agent produces or updates docstrings before invoking Sphinx.
 - The integration can be demonstrated with a test run.
-

Task 3 — Create Sphinx Skills for the Agent**Objective:**

Build reusable “skills” (commands, scripts, or tool wrappers) that the agent can call to automate Sphinx documentation generation.

What to do:

- Document each skill with its purpose, parameters, and expected output.

Deliverable:

A set of agent-usable Sphinx skills, with documentation and examples.

Acceptance criteria:

- The agent can successfully call the skills to generate a documentation site.
- The Sphinx build completes without errors or warnings for the sample project.